Project learning guide • November 2026

WorkFlow HR
Full-Stack Architecture

A code-based walkthrough of how the browser, JavaScript modules, Express API, MySQL database, sessions, and uploaded files fit together.

What you will understand

How a click in the UI becomes an HTTP request, how the API authenticates and authorizes it, how prepared SQL reads or writes MySQL, how the response updates the page, and where persistent data is stored.

This guide describes the project in this repository. It is not a generic HR-system design. Secrets and real passwords are intentionally excluded.

1. The application in one picture

WorkFlow HR is a same-origin web application: Express serves the browser files and its API from the same Node.js process. The browser does not connect to MySQL. Only the backend has database credentials.

```
PERSON
  │ uses
  ▼
Browser: index.html + styles.css + ES modules in src/
  │ fetch('/api/v1/...') + session cookie
  ▼
Express server: middleware → authentication → role/scope checks → route handler
  │ mysql2 prepared query / transaction
  ▼
MySQL: workflow_hr tables

Related persistence:
  express-session ── session ID cookie ── MySQLSessionStore ── web_sessions
  Uploaded documents ── server/uploads/ (private local filesystem in this version)
```

The browser keeps a temporary in-memory copy of the records currently displayed. MySQL is the durable source of truth. Refreshing the page reloads records from the API; it does not restore them from localStorage.

2. Technologies and what each one does

HTML5
Where it appears: index.html
Job in this project: Login form, page shell, navigation containers, modal containers, and browser entry point.

CSS
Where it appears: styles.css
Job in this project: Layout, responsive behavior, cards, forms, tables, modals, and visual states.

Vanilla JavaScript ES modules
Where it appears: src/
Job in this project: UI rendering, event handlers, client-side state, and calls to the API. No React, Vue, or frontend compiler is used.

Node.js 20+
Where it appears: server/src/
Job in this project: JavaScript runtime for the server, database scripts, password hashing, and uploads.

Express 4
Where it appears: server/src/index.js
Job in this project: HTTP server, static-file routes, API routes, middleware, and error handling.

mysql2/promise
Where it appears: server/src/db.js
Job in this project: Promise-based MySQL connection pool and parameterized SQL queries.

MySQL 8 / InnoDB
Where it appears: server/schema.sql
Job in this project: Durable relational records, unique keys, foreign keys, indexes, and transactions.

express-session
Where it appears: Express middleware
Job in this project: Stores a server-side login session; the browser receives only a session identifier cookie.

Custom MySQL session store
Where it appears: server/src/mysqlSessionStore.js
Job in this project: Stores and expires session records in the web_sessions table.

bcryptjs
Where it appears: Login and account creation
Job in this project: Hashes passwords with bcrypt; plaintext passwords are not stored in MySQL.

Zod
Where it appears: API write routes
Job in this project: Checks request fields, formats, and limits before the server writes data.

Helmet
Where it appears: Express middleware
Job in this project: Adds common HTTP security headers. This project explicitly disables Helmet's Content Security Policy.

express-rate-limit
Where it appears: Login route
Job in this project: Limits login attempts to 10 per 15-minute window per limiter key.

Multer
Where it appears: Payslip upload route
Job in this project: Parses multipart file uploads, accepts PDF uploads, and applies a configured size limit.

dotenv
Where it appears: Server startup scripts
Job in this project: Loads local values from server/.env. Production hosts should supply environment variables directly.

3. Frontend: what happens in the browser

3.1 Entry and modules

index.html loads styles.css and starts src/app.js as a native JavaScript module. The main module imports controllers, services, shared state, and HTML helpers. The browser renders pages by updating DOM elements; it is not a server-rendered React application.

Application entry
Responsibility: Registers browser globals needed by inline controls, binds authentication events, checks whether a session already exists.
Representative files: src/app.js

State
Responsibility: Holds current employee/leave/attendance data, session identity, and active page in memory.
Representative files: src/state/store.js

API client
Responsibility: Calls same-origin endpoints, includes cookies, parses JSON, handles errors and 401 sign-out behavior, refreshes bootstrap data.
Representative files: src/services/apiClient.js

Domain services
Responsibility: Small functions for login, employees, attendance, leave, and reports; they call the shared API client.
Representative files: src/services/*Service.js

Controllers
Responsibility: Connect clicks/forms to service calls and then render or refresh a view.
Representative files: src/controllers/*Controller.js

Views
Responsibility: Render common layout, dashboard, attendance, leave, employee, reports, tickets, and payslips UI.
Representative files: src/views/*View.js

Helpers
Responsibility: DOM lookup, safe text escaping, toast messages, formatting helpers.
Representative files: src/utils/helpers.js

3.2 Browser startup and login

1. The browser loads HTML, CSS, and JavaScript.

2. src/app.js calls GET /api/v1/auth/me.

3. If the server recognizes the session cookie, the UI loads GET /api/v1/bootstrap, fills in-memory state, and shows the app shell.

4. If there is no valid session, the login screen remains visible.

5. Submitting the login form calls POST /api/v1/auth/login. On success, bootstrap data is fetched and the UI renders navigation for the user's role.

3.3 Client state is a cache, not the database

state.data contains arrays such as employees, leaves, holidays, attendance, payslips, and tickets. A successful write usually calls refreshBootstrap() and re-renders. Closing or refreshing the browser loses this in-memory copy, but not committed MySQL data. Compatibility functions named saveData, saveUsers, and saveSession intentionally do nothing.

4. API and backend: how requests are handled

4.1 One normal request

```
Button/form event
  → controller calls a domain service
  → apiClient.fetch('/api/v1/...', { credentials: 'include' })
  → Express parses JSON or multipart form data
  → session middleware reads workflow.sid and loads web_sessions row
  → requireAuth reloads active user + employee from MySQL
  → attachDepartment loads the employee's department for scope checks
  → route validates input and checks role / record scope
  → mysql2 executes prepared SQL (or a transaction)
  → JSON response / file stream
  → service refreshes browser state and controller re-renders
```

4.2 Middleware order and common behavior

• Express disables the X-Powered-By header, applies Helmet headers, then parses JSON (up to 15 MB) and URL-encoded bodies (up to 2 MB).

• Session middleware is installed before application routes. The session cookie is named workflow.sid, HttpOnly, SameSite=Lax, expires after 8 hours, and is Secure in production.

• asyncRoute forwards rejected async handlers to common error middleware. API errors generally return JSON like {"error":{"message":"..."}}.

• fail() creates consistent API errors. Unmatched versioned API routes return 404.

• /api/health runs SELECT 1. It confirms the server can reach MySQL, but does not verify every schema table or complete business workflow.

4.3 API route map

GET /api/health
Purpose: Server/database health check.
Access notes: Public; checks a simple SQL query.

POST /api/v1/auth/login
Purpose: Validate email/password, create session, record login.
Access notes: Public; rate-limited; bcrypt comparison.

POST /api/v1/auth/logout
Purpose: Audit and destroy current session.
Access notes: Clears the session cookie.

GET /api/v1/auth/me
Purpose: Return current identity when session is valid.
Access notes: Authenticated.

GET /api/v1/bootstrap
Purpose: Initial client data: scoped employees, leaves, attendance, holidays, payslips, tickets.
Access notes: Authenticated; scope differs by role.

/api/v1/employees
Purpose: List, create, edit, deactivate employees/accounts.
Access notes: List is scoped; create/deactivate Admin only; edit checks role and manager department.

/api/v1/attendance
Purpose: Read attendance records and report data.
Access notes: Authenticated and role-scoped.

POST /api/v1/attendance/check-in|check-out
Purpose: Mark today's check-in or close an open record.
Access notes: Uses current employee identity from the session.

/api/v1/leaves
Purpose: List and submit leave requests.
Access notes: Rows are scoped; server recalculates working days.

POST /api/v1/leaves/:id/approve|reject
Purpose: Review a pending request.
Access notes: Manager/Admin; managers restricted to department.

/api/v1/holidays
Purpose: List company holidays; add a holiday.
Access notes: Add is Admin only.

/api/v1/tickets
Purpose: List/create tickets; change ticket status.
Access notes: List scoped; status change Manager/Admin with department check.

/api/v1/payslips
Purpose: List metadata, upload PDF, download, or delete.
Access notes: Downloads verify employee ownership; upload/delete Admin only.

GET /api/v1/dashboard/summary
Purpose: Counts active employees, today's attendance, pending leave, holidays.
Access notes: Role-scoped where applicable.

GET /api/v1/reports/attendance?month=YYYY-MM
Purpose: Return month's attendance rows.
Access notes: Role-scoped; month format validated.

5. Authentication, sessions, roles, and security

Password and session lifecycle

1. The seed script or Admin employee-creation route hashes a password using bcryptjs with cost factor 12, then writes only the hash into users.password_hash.

2. Login looks up a user by normalized email, requires both active user and active employee, then uses bcrypt.compare.

3. After success, Express regenerates the session ID to reduce session-fixation risk and stores the user's numeric users.id as req.session.userId.

4. The browser receives only an HttpOnly session cookie. The session body remains server-side in web_sessions.

5. On protected API requests the server reloads the user and employee; a disabled user or inactive employee is rejected even if an old cookie exists.

6. Logout destroys the database session and clears the cookie.

Roles and scope

Employee
Typical visibility/actions: Own employee profile, attendance, leave, tickets, and payslips.
Where enforced: Backend SQL filters by current employee ID; UI also hides management navigation.

Manager
Typical visibility/actions: Department-level team records and approvals; cannot manage the whole company.
Where enforced: Backend checks department ID on reads and protected writes.

Admin
Typical visibility/actions: Company-wide management, accounts, holidays, payslip uploads and deletes.
Where enforced: Backend role middleware and per-route rules.

Other protections include parameterized SQL, Zod request validation, a login rate limit, upload type/size checks, a private uploads directory, production session-secret length enforcement, and Helmet headers. The Content Security Policy is disabled in code, so it is not one of the protections currently provided.

6. MySQL data model and relationships

The application uses raw SQL with mysql2, not an ORM. IDs are generally BIGINT UNSIGNED. The API converts database rows into smaller browser-facing objects so internal columns such as password hashes are not returned.

```
departments 1 ─── * employees 1 ─── 1 users
                          │                  │
                          ├── * attendance   │
                          ├── * leave_requests (reviewer_id → users.id)
                          ├── * payslips       (uploaded_by → users.id)
                          ├── * tickets        (assigned_to → users.id)
                          └── * audit_logs     (actor_user_id → users.id)

holidays.created_by ──────────────────→ users.id
web_sessions is separate: session_id → serialized session body + expiry
```

departments
What it stores: Unique department names and creation time.
Key relationships / rules: One department can have many employees.

employees
What it stores: Employee code, identity/contact fields, department, business role/status, leave balance, timestamps.
Key relationships / rules: Employee code and email unique; department foreign key; status Active/Inactive.

users
What it stores: Login email, password hash, role, active flag, last login.
Key relationships / rules: One user per employee; separate from HR employee profile.

web_sessions
What it stores: Session ID, expiry timestamp, serialized session data.
Key relationships / rules: Used by Express session middleware; expired rows are periodically deleted.

attendance
What it stores: Employee, work date, status, check-in/out timestamps.
Key relationships / rules: Unique employee/date prevents duplicate daily check-ins.

holidays
What it stores: Unique holiday date, name, creator user, creation time.
Key relationships / rules: Holiday dates are used while calculating working leave days.

leave_requests
What it stores: Leave type/date range, working/paid/LOP days, reason, optional document metadata, status, reviewer.
Key relationships / rules: Employee and reviewer foreign keys; status Pending/Approved/Rejected.

payslips
What it stores: Employee, month, private file storage key, original filename, uploader.
Key relationships / rules: One payslip per employee/month; PDF itself is not stored in MySQL.

tickets
What it stores: Employee issue subject/description, priority, status, assignee and timestamps.
Key relationships / rules: Priority Low/Normal/High; status Open/In Progress/Resolved.

audit_logs
What it stores: Actor, action, entity, entity ID, optional JSON details, time.
Key relationships / rules: Records significant actions such as login, leave review, payslip changes.

Two different employee identifiers

MySQL uses numeric employees.id for foreign keys. The UI/API commonly displays the human-readable employee_code such as EMP001. Keep them distinct: the API converts DB IDs into employee codes for employee lists, while attendance/payslip rows have their own numeric record IDs.

7. The main business flows

Employee creation

An Admin submits an employee form. The API validates fields, begins a MySQL transaction, creates or reuses the department, inserts the employee, hashes the initial password, inserts the linked login user, and commits. Duplicate employee code or email returns a conflict response. A failed transaction rolls back both records.

Attendance

The server computes the current calendar date in Asia/Kolkata. It marks a check-in Late after 09:45 India local time; otherwise Present. The persisted event timestamp is UTC. A unique employee/date key prevents a second check-in. Check-out updates only an existing open row for today. API view helpers convert dates and times for the browser.

Leave application and approval

1. The browser can preview working days using weekdays and the currently loaded holiday list.

2. The server independently validates the leave type, dates, reason, and document. It queries holidays in the requested range and recalculates the weekday count; the client is not trusted for the final count.

3. The server compares requested days with the current leave balance. Paid days and loss-of-pay (LOP) days are stored separately. A Paid Leave request with insufficient balance is rejected.

4. Sick leave longer than three working days requires a PDF/PNG/JPEG. The server checks the file signature as well as the data URL type and writes the document to the private uploads folder.

5. A Manager/Admin approval starts a transaction, locks the leave row with FOR UPDATE, checks Pending status and department scope, deducts paid days only on approval, updates reviewer/status, then commits.

Holidays, tickets, reports, payslips

• Holidays: Admin creates a unique date; holidays affect leave-day calculations.

• Tickets: an employee creates a ticket; Manager/Admin can change its status, with managers limited to their department.

• Attendance reports: server returns month-filtered rows and role scope; browser builds a downloadable Excel-compatible HTML file with an .xls extension.

• Leave reports: built in the browser from bootstrap data. Manager scope is assembled client-side using employee names/departments in the loaded data.

• Payslips: Admin uploads a PDF; Multer stores it in server/uploads/payslips, the server checks the PDF signature, and MySQL stores its metadata/path. Downloads check that a non-Admin owns the payslip before sending it. The uploads directory must never be exposed as public static content.

8. How setup and startup work

Install
Command/action: npm install
What it does: Installs dependencies from package.json/package-lock.json.

Configure
Command/action: Copy server/.env.example to server/.env
What it does: Supplies local port, session secret, MySQL host/database/users/passwords, upload location and size limit.

Start MySQL
Command/action: Docker Compose or installed MySQL 8
What it does: MySQL must listen at the host/port in server/.env.

Create schema
Command/action: npm run db:migrate
What it does: Reads server/schema.sql and runs its CREATE DATABASE/USE/CREATE TABLE statements using DB_ADMIN credentials.

First account
Command/action: npm run db:seed-admin -- EMP001 "Name" admin@example.com Administration
What it does: Prompts for a hidden 12+ character password; inserts department, employee and hashed Admin login.

Run server
Command/action: npm start
What it does: Loads server/.env, creates upload folder, installs middleware/routes, and listens on PORT or 3000.

Open app
Command/action: http://localhost:3000
What it does: Browser downloads static frontend files from Express.

PowerShell may use npm.cmd if the script policy blocks npm. Run SQL statements in MySQL Workbench, not in a normal PowerShell or Bash prompt.

9. Configuration and important boundaries

• PORT: server listening port; default 3000.

• APP_ORIGIN: documented local/production origin value; inspect route code before assuming every environment variable is actively used.

• SESSION_SECRET: signs session cookies. Production requires at least 32 characters.

• DB_HOST, DB_PORT, DB_NAME, DB_USER, DB_PASSWORD: runtime MySQL connection.

• DB_ADMIN_USER, DB_ADMIN_PASSWORD: elevated credentials used by migration and must be protected.

• UPLOAD_DIR, MAX_UPLOAD_MB: local private storage location and payslip upload limit.

• .env and uploaded files are ignored by Git. Never commit production secrets, real payroll files, or local DB credentials.

10. Source-code map

index.html, styles.css
Read it when you want to understand…: Browser shell and visual styling.

src/app.js
Read it when you want to understand…: Frontend startup, global event wiring, holiday/report actions.

src/services/apiClient.js
Read it when you want to understand…: HTTP requests, cookies, JSON/error handling, bootstrap refresh.

src/controllers/
Read it when you want to understand…: UI actions and page behavior.

src/services/
Read it when you want to understand…: Frontend domain operations that call API endpoints.

src/views/
Read it when you want to understand…: Reusable layout and feature view rendering.

src/state/store.js
Read it when you want to understand…: In-memory UI state; no DB persistence.

server/src/index.js
Read it when you want to understand…: Express middleware, authentication, every API route, business rules, upload handling, and static routes.

server/src/db.js
Read it when you want to understand…: MySQL pool and audit insert helper.

server/src/mysqlSessionStore.js
Read it when you want to understand…: Database-backed Express session get/set/delete/expiry cleanup.

server/schema.sql
Read it when you want to understand…: Database schema, indexes, unique constraints, and foreign keys.

server/src/migrate.js
Read it when you want to understand…: Applies the initial schema from the command line.

server/src/seed-admin.js
Read it when you want to understand…: Creates the first Admin identity and hashes its password.

package.json
Read it when you want to understand…: Runtime dependencies and start/check/migration/seed scripts.

11. Troubleshooting by symptom

EADDRINUSE :::3000
Meaning: Another process already owns port 3000. Often that is the app already running.
Next check: Open localhost:3000. To restart, stop the old server with Ctrl+C in its terminal before starting another.

ECONNREFUSED 127.0.0.1:3306
Meaning: No MySQL service accepted the connection at that address/port.
Next check: Check MySQL/Docker is running, listening on 3306, and DB_HOST/DB_PORT are correct.

ER_ACCESS_DENIED_ERROR
Meaning: MySQL rejected the username/password or host grant.
Next check: Check DB_USER/DB_PASSWORD or DB_ADMIN_USER/DB_ADMIN_PASSWORD separately; do not paste secrets into chat.

Table ... does not exist
Meaning: The schema migration did not finish, or app points at another database.
Next check: Run db:migrate with the correct DB_ADMIN connection; verify the Workbench schema is workflow_hr.

Foreign-key columns incompatible
Meaning: An existing table was created with an older/different column type.
Next check: Compare SHOW CREATE TABLE with server/schema.sql. Back up/rename legacy tables before recreating the app schema.

Duplicate entry ... employee_code
Meaning: The employee code already exists; the seed script always inserts a new employee.
Next check: Inspect existing employees/users before choosing a unique code. A duplicate seed attempt does not reset a password.

Login rejected
Meaning: Email/password mismatch, inactive user, inactive employee, or wrong database.
Next check: Check user is_active, employee status, and email; passwords are bcrypt hashes and cannot be read back.

Page loads but API fails
Meaning: Static frontend can load even when DB/schema/session access is broken.
Next check: Open /api/health, then inspect server terminal logs and session/database configuration.

12. Learning path: trace one feature end to end

1. Pick a button in a view/controller, for example check-in in src/controllers/dashboardController.js.
2. Follow the service it calls, such as src/services/attendanceService.js.
3. Find the API path passed to api() and note the HTTP method/body.
4. Find the matching Express route in server/src/index.js.
5. Trace its middleware: session, authentication, department attachment, and role checks.
6. Read the SQL statement and identify its table, key, and transaction behavior.
7. Find the JSON response shape; compare it with the frontend state/view fields.
8. Check the schema constraint that protects the data (for example, the unique employee/date attendance key).

Glossary

Frontend
Plain-language meaning: Code running in the user's browser: HTML, CSS, and JavaScript.

Backend
Plain-language meaning: Server-side Node.js/Express code that receives requests and applies rules.

API
Plain-language meaning: A defined set of HTTP routes the browser uses to ask the backend to read/change data.

Middleware
Plain-language meaning: Reusable code that runs before a route, for example body parsing or session authentication.

Session
Plain-language meaning: Server-held sign-in state associated with a browser cookie.

Prepared query
Plain-language meaning: SQL where values are supplied separately from SQL text, reducing SQL injection risk.

Foreign key
Plain-language meaning: A database rule that keeps a reference pointing to a real row in another table.

Transaction
Plain-language meaning: A group of database changes committed together or rolled back together.

LOP
Plain-language meaning: Loss of pay days: working days requested beyond paid leave balance.

Source of truth

Plain-language meaning: The system holding the durable authoritative value; here it is MySQL for business records.

Prepared from the repository’s current code and README. Validation snapshot: syntax check passed, initial MySQL migration succeeded, and the local homepage/database health endpoint responded successfully. The project’s README lists tickets/policies and several HR policy decisions as unfinished scope.